

实验名称 实验2 复杂网络基本拓扑性质分析

一、实验内容及结果

- 1、选择 5 种不同类型的复杂网络（必须包含一种真实网络，一种有向网络，一种加权网络），计算节点的度，节点的入度和出度，网络的平均度，画出度分布图
- 2、计算这 5 种网络的直径、平均路径长度、节点的聚类系数、网络的聚类系数等参数

源代码：

网络一：

Task1:

```
# 无向网络-K 近邻耦合图
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

n=10
G=nx.watts_strogatz_graph(n,6,0.0) #k=6
plt.figure(figsize=(4,4))
nx.draw(G,pos=nx.circular_layout(G),node_size=700,node_color="green",with_labels=True,width=3)

d=dict(nx.degree(G))
print("节点的度: :",d)
print("网络的平均度为: ",sum(d.values())/len(G.nodes))
nx.degree_histogram(G)
x=list(range(max(d.values())+1))
y=[i/len(G.nodes) for i in nx.degree_histogram(G)]
plt.bar(x,y,width=0.5,color='red')
plt.xlabel("$k$")
plt.ylabel("$P_{(k)}$")
plt.show()
```

Task2:

```
print("网络直径为: ",nx.diameter(G))
print("节点 1 和 7 的最短路径",nx.shortest_path(G,source=1,target=7))
print("节点 1 和 7 的所有最短路径",list(nx.all_shortest_paths(G,source=1,target=7)))
```

```

print("节点 1 和 7 的最短路径长度",nx.shortest_path_length(G,source=1,target=7))
print("节点 1 和 7 的效率为",nx. efficiency(G,1,7))
print("网络的平均路径长度",nx.average_shortest_path_length(G))
print("网络的密度为: ",nx.density(G))

```

```

print("节点的聚类系数",nx.clustering(G))
print("平均聚类系数",nx.average_clustering(G))
print("全局聚类系数",nx.transitivity(G))
print("节点横截性",nx.betweenness_centrality(G))
node_bc=nx.betweenness_centrality(G)
print("网络横截性",sum(node_bc.values())/len(node_bc))

```

网络二:

Task1:

```

df=pd.read_csv('d:/Netdata/save/net1.csv')
G=nx.from_pandas_edgelist(df,'source','target','weight',create_using =nx.
Graph())
w=[G[e[0]][e[1]]['weight'] for e in G.edges()]
pos = nx.spring_layout(G) # 设置网络的布局
nx.draw(G, pos, with_labels=True, node_color="green", node_size=700, edge
_color="black", width=w)
edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
plt.show()

```

```

print("网络节点的加权重度:")
print(nx.degree(G,weight='weight'),end='\n')
print("网络的边度:")
for e in G.edges():
    print(G[e[0]][e[1]]['weight'],end=' ')
d=dict(nx.degree(G,weight='weight'))
print("\n 网络平均度为: ",sum(d.values())/len(G.edges))

```

```

def get_pdf(G):
    all_k=[G.degree(i,weight='weight') for i in G.nodes()]
    k=list(set(all_k))
    N=len(G.nodes())

    Pk=[]
    for ki in sorted(k):
        c=0
        for i in G.nodes():

```

```

        if G.degree(i,weight='weight')==ki:
            c+=1
        Pk.append(c/N)

    return sorted(k),Pk

```

```

mid_d,p=get_pdf(G)
plt.bar(mid_d,p,width=0.4,color='red')
plt.xlabel("$k$")
plt.ylabel("$P_{(k)}$")
plt.show()

```

Task2:

```

def w_efficiency(G,a,b):
    shortest_pl=nx.shortest_path_length(G,source=a,target=b,weight='weight')
    w_efficiency=1/shortest_pl
    return w_efficiency

```

```

def w_diameter(G):
    if nx.is_connected(G):
        length = dict(nx.all_pairs_dijkstra_path_length(G))
        max_length = 0
        for n in length.values():
            max_length = max(max(max_length, val) for val in n.values())

        return max_length
    else:
        return print("图不是连通的，无法计算无向加权直径。")

```

```

print("网络直径为: ",w_diameter(G))
print("节点 B 和 F 的最短路径",nx.shortest_path(G,source='B',target='F',weight='weight'))
print("节点 B 和 F 的所有最短路径",list(nx.all_shortest_paths(G,source='B',target='F',weight='weight')))
print("节点 B 和 F 的最短路径长度",nx.shortest_path_length(G,source='B',target='F',weight='weight'))
print("节点 B 和 F 的效率为",w_efficiency(G,'B','F'))
print("网络的平均路径长度",nx.average_shortest_path_length(G,weight='weight'))
print("网络的密度为: ",nx.density(G))

```

```

print("节点的聚类系数",nx.clustering(G,weight='weight'))
print("平均聚类系数",nx.average_clustering(G,weight='weight'))

```

```

print("节点横截性",nx.betweenness centrality(G,weight='weight'))
node_bc=nx.betweenness centrality(G)
print("网络横截性",sum(node_bc.values())/len(node_bc))

```

网络三:

Task1:

```

df=pd.read_csv('d:/Netdata/save/net2.csv')
DG=nx.from_pandas_edgelist(df,'source','target','weight',create_using =nx
.DiGraph())
w=[DG[e[0]][e[1]]['weight'] for e in DG.edges()]
#nx.draw(G,with_labels=True,node_color="green",node_size=700,edge_color="
black",width=w)
pos=nx.spring_layout(DG,seed=700)

nx.draw_networkx_nodes(DG, pos, node_size=700,node_color="green")
nx.draw_networkx_labels(DG, pos, font_size=12, font_family="sans-
serif")
nx.draw_networkx_edges(DG,pos,width=3)
edge_labels=nx.get_edge_attributes(DG,"weight")
nx.draw_networkx_edge_labels(DG,pos,edge_labels)
plt.show()

```

```

print("节点的入度",DG.in_degree(DG,weight='weight'),end='\n')
print("节点的出度",DG.out_degree(DG,weight='weight'),end='\n')
d1=dict(DG.in_degree(DG,weight='weight'))
print('网络的平均入度为: ',sum(d1.values())/len(G.nodes))
d2=dict(DG.out_degree(DG,weight='weight'))
print('网络的平均出度为: ',sum(d2.values())/len(G.nodes))
d3=dict(DG.degree(DG,weight='weight'))
print('网络的平均度为: ',sum(d3.values())/len(G.nodes))

```

```

def in_degree(d1):
    in_degree_count = {}
    for degree in d1:
        if degree not in in_degree_count:
            in_degree_count[degree] = 1
        else:
            in_degree_count[degree] += 1

    total = sum(in_degree_count.values())
    for degree in in_degree_count:
        in_degree_count[degree] /= total

```

```

degrees = list(d1.values())
frequencies = list(in_degree_count.values())

plt.figure(figsize=(10, 6))
plt.bar(degrees, frequencies, width=0.1, color='red')
plt.xlabel('Weighted In-Degree')
plt.ylabel('Proportion')
plt.show()
return degrees

def out_degree(d1):
    in_degree_count = {}
    for degree in d1:
        if degree not in in_degree_count:
            in_degree_count[degree] = 1
        else:
            in_degree_count[degree] += 1

    total = sum(in_degree_count.values())
    for degree in in_degree_count:
        in_degree_count[degree] /= total

    degrees = list(d1.values())
    frequencies = list(in_degree_count.values())

    plt.figure(figsize=(10, 6))
    plt.bar(degrees, frequencies, width=0.1, color='blue')
    plt.xlabel('Weighted Out-Degree')
    plt.ylabel('Proportion')
    plt.show()
    return degrees

```

```

in_degree(d1)
out_degree(d2)

```

Task2:

```

print("网络是否强连通: ", nx.is_strongly_connected(DG))
print("节点 1 和 4 的最短路径",
      nx.shortest_path(DG, source=1, target=4, weight='weight'))
print("节点 1 和 4 的最短路径长度",
      nx.shortest_path_length(DG, source=1, target=4, weight='weight'))
print("节点 1 和 4 的效率为", w_efficiency(DG, 1, 4))
print("网络的密度为: ", nx.density(DG))

print("节点的聚类系数", nx.clustering(DG, weight='weight'))

```

```

print("平均聚类系数",nx.average_clustering(DG,weight='weight'))
print("节点横截性",nx.betweenness centrality(DG,weight='weight'))
node_bc=nx.betweenness centrality(DG)
print("网络横截性",sum(node_bc.values())/len(node_bc))

```

网络四:

Task1:

```

G=nx.karate_club_graph()
nx.draw(G,with_labels=True,node_color="green")
plt.show()

d=dict(nx.degree(G))
print('节点的度: ',d)
print('网络平均度: ',sum(d.values())/len(G.nodes))

nx.degree_histogram(G)
x=list(range(max(d.values())+1))
y=[i/len(G.nodes) for i in nx.degree_histogram(G)]
plt.bar(x,y,width=0.5,color='red')
plt.xlabel("$k$")
plt.ylabel("$P(k)$")
plt.show()

```

Task2:

```

print("网络直径为: ",nx.diameter(G))
print("节点 0 和 33 的最短路径",nx.shortest_path(G,source=0,target=33))
print("节点 0 和 33 的所有最短路径",list(nx.all_shortest_paths(G,source=0,target=33)))
print("节点 0 和 33 的最短路径长度",nx.shortest_path_length(G,source=0,target=33))
print("节点 0 和 33 的效率为",nx. efficiency(G,0,33))
print("网络的平均路径长度",nx.average_shortest_path_length(G))
print("网络的密度为: ",nx.density(G))

print("节点的聚类系数",nx.clustering(G))
print("平均聚类系数",nx.average_clustering(G))
print("全局聚类系数",nx.transitivity(G))
print("节点横截性",nx.betweenness centrality(G))
node_bc=nx.betweenness centrality(G)
print("网络横截性",sum(node_bc.values())/len(node_bc))

```

网络五:

Task1:

```
import networkx as nx
import matplotlib.pyplot as plt
G=nx.grid_graph((6,6),periodic=False)
plt.figure(figsize=(4,4))
nx.draw(G,node_size=50,node_color="green",with_labels=False)

d=dict(nx.degree(G))
print('节点的度: ',d)
print('网络平均度: ',sum(d.values())/len(G.nodes))
nx.degree_histogram(G)
x=list(range(max(d.values())+1))
y=[i/len(G.nodes) for i in nx.degree_histogram(G)]
plt.bar(x,y,width=0.5,color='red')
plt.xlabel("$k$")
plt.ylabel("$P_{(k)}$")
plt.show()
```

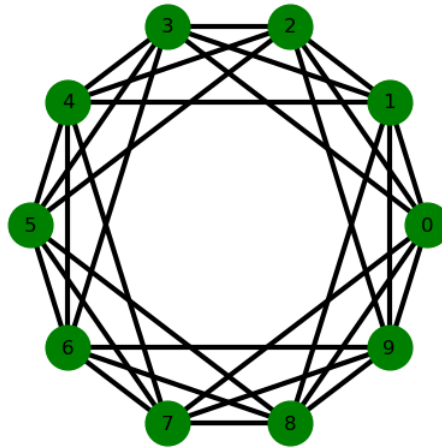
Task2:

```
print("网络直径为: ",nx.diameter(G))
print("网络的平均路径长度",nx.average_shortest_path_length(G))
print("网络的密度为: ",nx.density(G))

print("平均聚类系数",nx.average_clustering(G))
print("全局聚类系数",nx.transitivity(G))
print("节点横截性",nx.betweenness centrality(G))
node_bc=nx.betweenness centrality(G)
print("网络横截性",sum(node_bc.values())/len(node_bc))
```

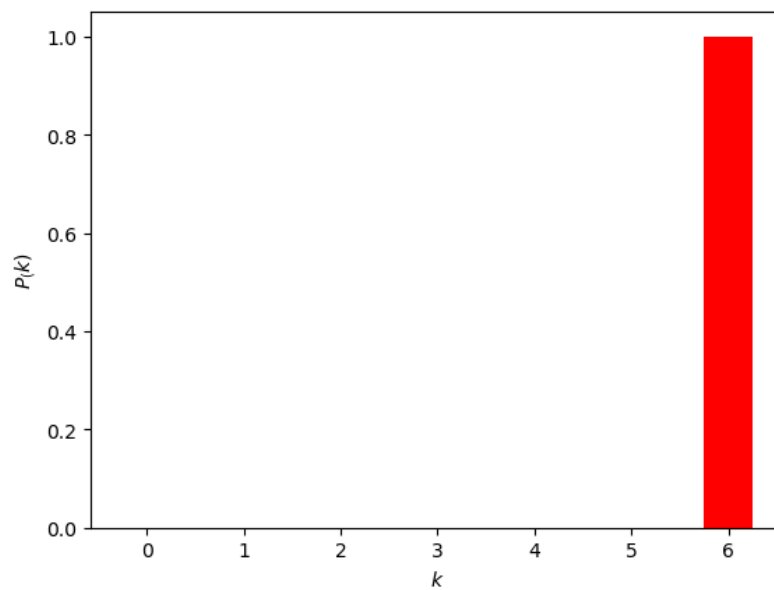
运行结果 (截图):

网络一, K 近邻耦合图 (k=6), 无向无权:



Task1:

节点的度: : {0: 6, 1: 6, 2: 6, 3: 6, 4: 6, 5: 6, 6: 6, 7: 6, 8: 6, 9: 6}
网络的平均度为: 6.0



K 近邻耦合图中每个节点都与 k 个节点相连, 所以每个节点度都是 k (6)。

Task2:

网络直径为: 2

节点1和7的最短路径 [1, 8, 7]

节点1和7的所有最短路径 [[1, 0, 7], [1, 9, 7], [1, 4, 7], [1, 8, 7]]

节点1和7的最短路径长度 2

节点1和7的效率为 0.5

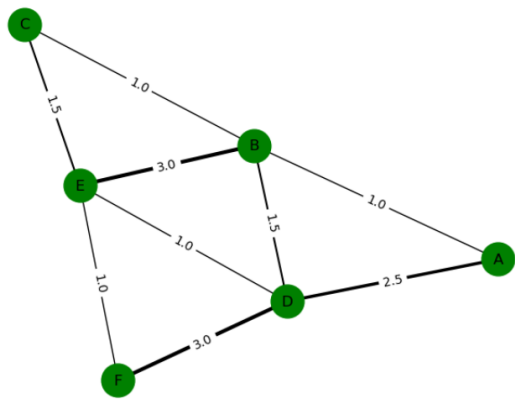
网络的平均路径长度 1.3333333333333333

网络的密度为: 0.6666666666666666

节点的聚类系数 {0: 0.6, 1: 0.6, 2: 0.6, 3: 0.6, 4: 0.6, 5: 0.6, 6: 0.6, 7: 0.6, 8: 0.6, 9: 0.6}
平均聚类系数 0.5999999999999999
全局聚类系数 0.6
节点横截性 {0: 0.041666666666666664, 1: 0.041666666666666664, 2: 0.041666666666666664, 3: 0.041666666666666664, 4: 0.041666666666666664, 5: 0.041666666666666664, 6: 0.041666666666666664, 7: 0.041666666666666664, 8: 0.041666666666666664, 9: 0.041666666666666664}
网络横截性 0.041666666666666667

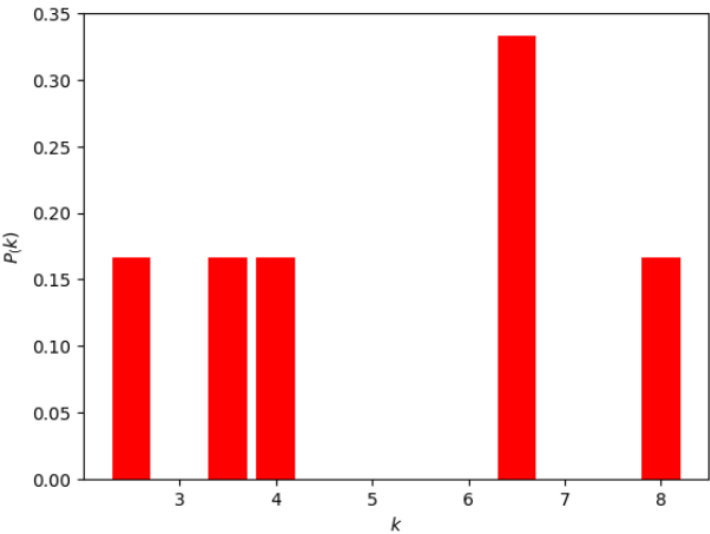
κ 近邻耦合图节点的度、聚类系数、横截性均相同，因为网络中每个节点连接性质相同。

网络二，无向加权图：



Task1:

网络节点的加权重度：
[('A', 3.5), ('B', 6.5), ('D', 8.0), ('C', 2.5), ('E', 6.5), ('F', 4.0)]
网络的边度：
1.0 2.5 1.5 1.0 3.0 3.0 1.0 1.5 1.0
网络平均度为： 3.4444444444444446



加权网络节点的度是与之相连的边的度之和。

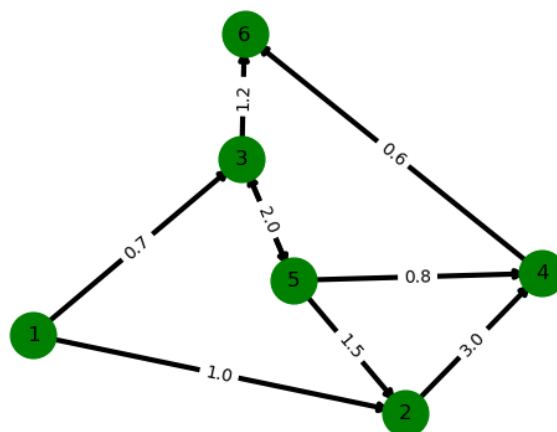
Task2:

网络直径为： 4.5
节点B和F的最短路径 ['B', 'C', 'E', 'F']
节点B和F的所有的最短路径 [['B', 'C', 'E', 'F'], ['B', 'D', 'E', 'F']]
节点B和F的最短路径长度 3.5
节点B和F的效率为 0.2857142857142857
网络的平均路径长度 2.1666666666666665

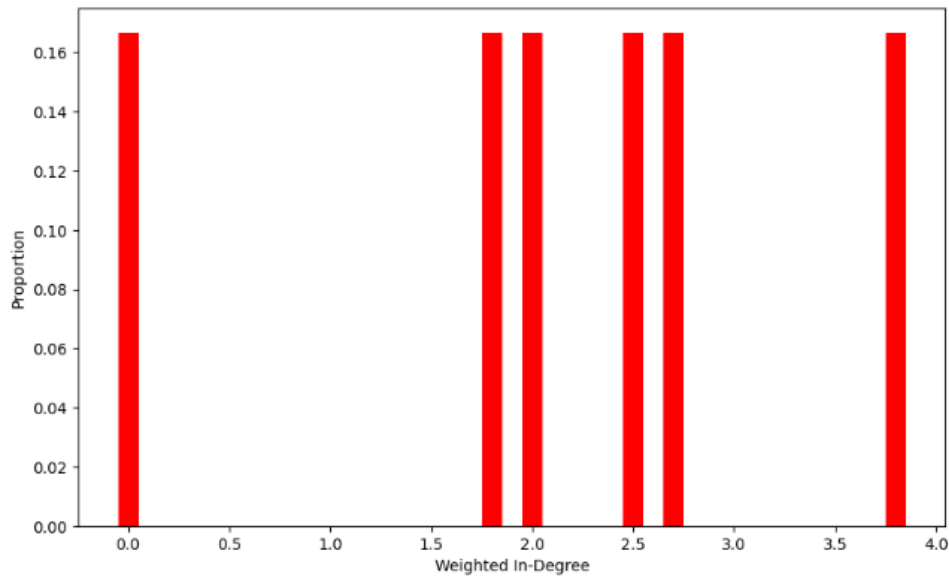
节点的聚类系数 {'A': 0.5178720843256432, 'B': 0.26975241677064205, 'D': 0.2581571915406473, 'C': 0.5503212081491045, 'E': 0.7498567691361}
平均聚类系数 0.3900696893999551
节点横截性 {'A': 0.0, 'B': 0.3333333333333333, 'D': 0.2333333333333333, 'C': 0.1666666666666666, 'E': 0.45, 'F': 0.0}
网络横截性 0.10000000000000002

加权网络中，许多网络性质都要考虑权重的影响，如网络直径、节点效率等，都不能直接使用 `networkx` 中的函数，需要另定义有权网络下的函数。

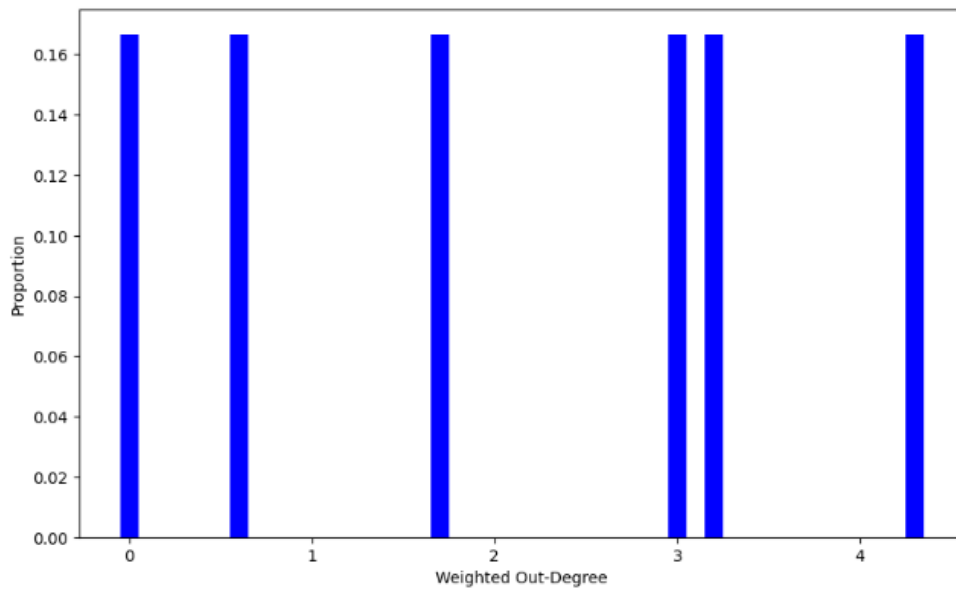
网络三，有向加权图：



Task1:
节点的入度 [(1, 0), (2, 2.5), (3, 2.7), (4, 3.8), (5, 2.0), (6, 1.7999999999999998)]
节点的出度 [(1, 1.7), (2, 3.0), (3, 3.2), (4, 0.6), (5, 4.3), (6, 0)]
网络的平均入度为： 2.1333333333333333
网络的平均出度为： 2.1333333333333333
网络的平均度为： 4.2666666666666667



[0, 2.5, 2.7, 3.8, 2.0, 1.7999999999999998]



[1.7, 3.0, 3.2, 0.6, 4.3, 0]

有向加权网络的度分布包括每个节点的入度分布与出度分布。

Task2:

网络是否强连通: **False**

节点1和4的最短路径 **[1, 3, 5, 4]**

节点1和4的最短路径长度 **3.5**

节点1和4的效率为 **0.2857142857142857**

网络的密度为: **0.3**

节点的聚类系数 {1: 0, 2: 0.08514549248817256, 3: 0, 4: 0.08514549248817256, 5: 0.05108729549290354, 6: 0}

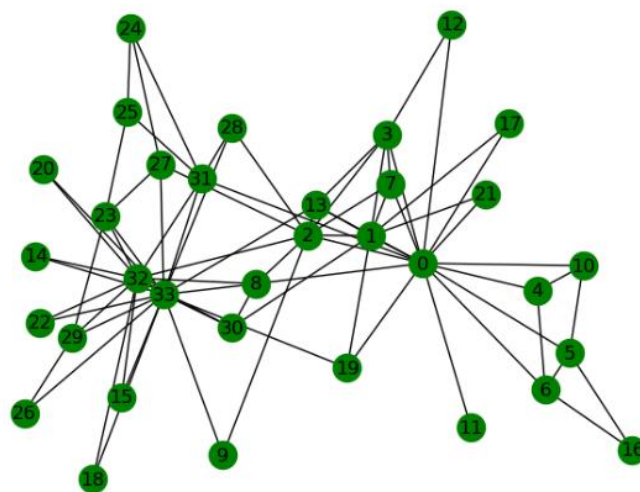
平均聚类系数 **0.03689638007820811**

节点横截性 {1: 0.0, 2: 0.0, 3: 0.15000000000000002, 4: 0.1, 5: 0.15000000000000002, 6: 0.0}

网络横截性 **0.05833333333333333**

有向加权网络中, 计算网络直径和网络平均最短路径的前提是网络是强连通的。

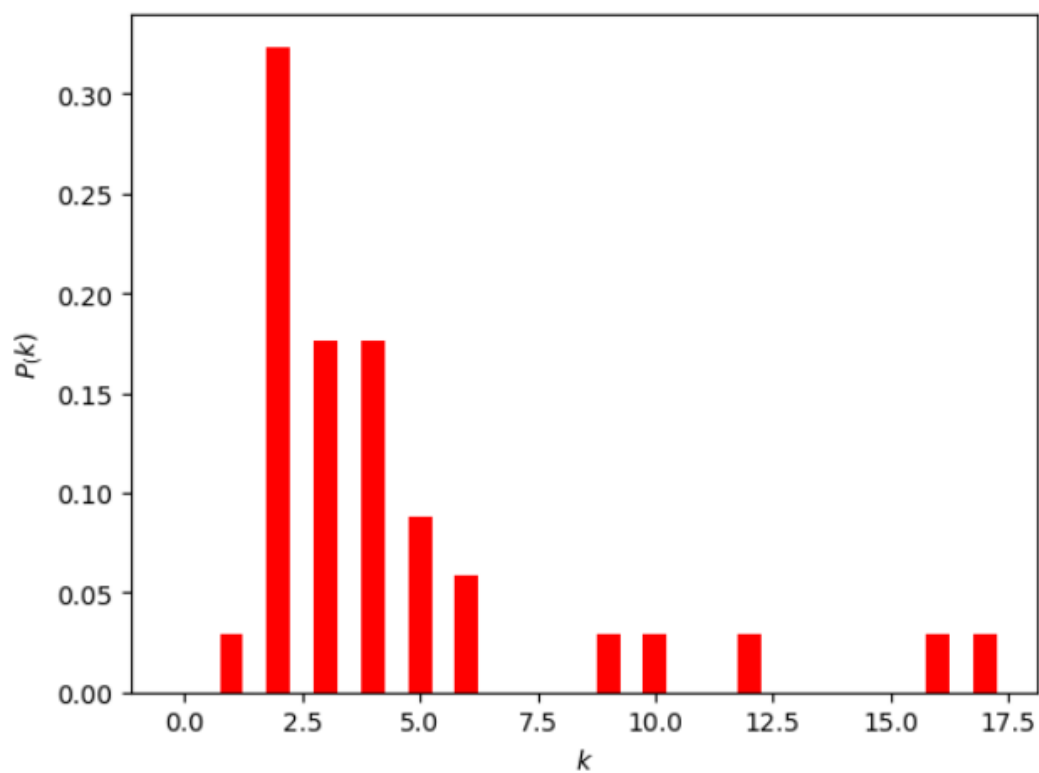
网络四，真实网络，karate_club:



Task1:

节点的度: {0: 16, 1: 9, 2: 10, 3: 6, 4: 3, 5: 4, 6: 4, 7: 4, 8: 5, 9: 2, 10: 3, 11: 1, 12: 2, 13: 5, 14: 2, 15: 2, 16: 2, 17: 2, 18: 2, 19: 3, 20: 2, 21: 2, 22: 2, 23: 5, 24: 3, 25: 3, 26: 2, 27: 4, 28: 3, 29: 4, 30: 4, 31: 6, 32: 12, 33: 17}
网络平均度: 4.588235294117647

9: 2, 10: 3, 11: 1, 12: 2, 13: 5, 14: 2, 15: 2, 16: 2, 17: 2, 18: 2, 19: 3, 20: 2, 21: 2, 22: 2, 23: 5, 24: 3, 25: 3, 26: 2, 27: 4, 28: 3, 29: 4, 30: 4, 31: 6, 32: 12, 33: 17}



karate_club 网络中，节点 0 和节点 33 属于中心节点，两者拥有最大的度。

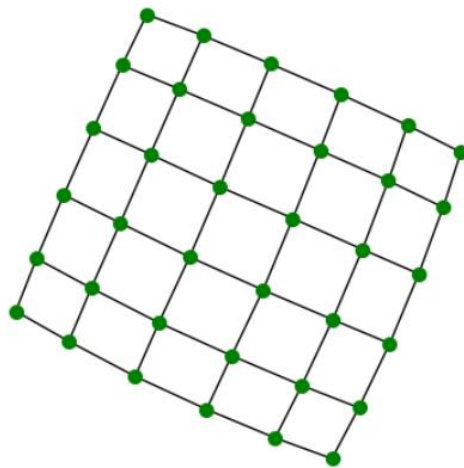
Task2:

网络直径为： 5
节点0和33的最短路径 [0, 8, 33]
节点0和33的所有的最短路径 [[0, 8, 33], [0, 13, 33], [0, 19, 33], [0, 31, 33]]
节点0和33的最短路径长度 2
节点0和33的效率为 0.5
网络的平均路径长度 2.408199643493761
网络的密度为： 0.13903743315508021

节点的聚类系数 {0: 0.15, 1: 0.3333333333333333, 2: 0.2444444444444444, 3: 0.6666666666666666, 4: 0.6666666666666666, 5: 0.5, 6: 0.5, 7: 1.0, 8: 0.5, 9: 0, 10: 0.6666666666666666, 11: 0, 12: 1.0, 13: 0.6, 14: 1.0, 15: 1.0, 16: 1.0, 17: 1.0, 18: 1.0, 19: 0.3333333333333333, 20: 1.0, 21: 1.0, 22: 1.0, 23: 0.4, 24: 0.3333333333333333, 25: 0.3333333333333333, 26: 1.0, 27: 0.1666666666666666, 28: 0.3333333333333333, 29: 0.6666666666666666, 30: 0.5, 31: 0, 32: 0.1969696969696969, 33: 0.11029411764705882}
平均聚类系数 0.5706384782076823
全局聚类系数 0.2556818181818182
节点横截性 {0: 0.43763528138528146, 1: 0.053936688311688304, 2: 0.14365680615680618, 3: 0.011909271284271283, 4: 0.0006313131313131313, 5: 0.02998737373737374, 6: 0.029987373737373736, 7: 0.0, 8: 0.05592682780182781, 9: 0.0008477633477633478, 10: 0.0006313131313131313, 11: 0.0, 12: 0.0, 13: 0.04586339586339586, 14: 0.0, 15: 0.0, 16: 0.0, 17: 0.0, 18: 0.0, 19: 0.03247504810004811, 20: 0.0, 21: 0.0, 22: 0.0, 23: 0.017613636363636363, 24: 0.0022095959595959595, 25: 0.0038404882154882154, 26: 0.0, 27: 0.02233345358345358, 28: 0.0017947330447330447, 29: 0.0029220779220779218, 30: 0.014411976911976909, 31: 0.13827561327561325, 32: 0.145247113997114, 33: 0.30407497594997596}
网络横截性 0.044006238859180036

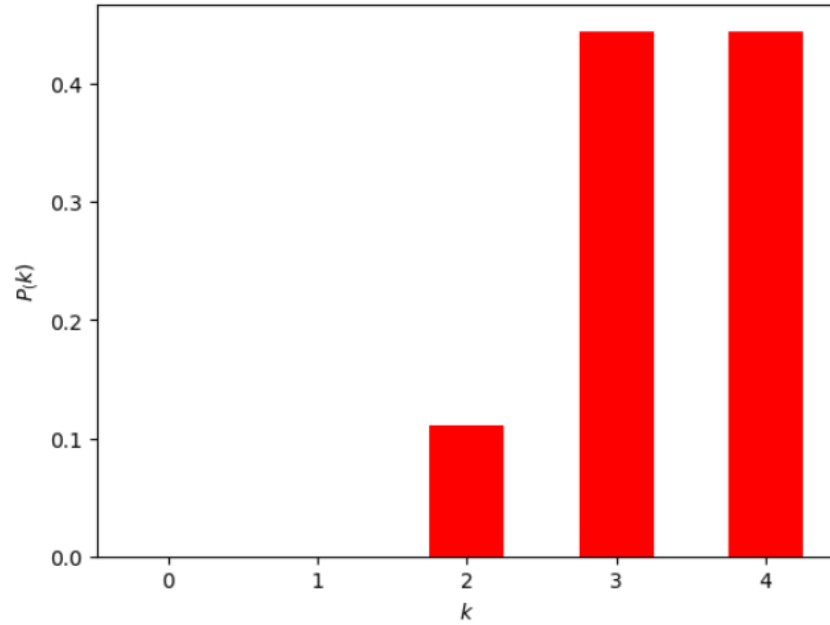
节点 0 和节点 33 的聚类系数明显小于其它点，因为它们邻居节点数多，各个邻居互为邻居的概率小；同时这两个节点横截性高于其它点，因为它们分属两个社群的中心，通过它们的最短路径数目多。

网络五，二维方格图：



Task1:

节点的度: {(0, 0): 2, (0, 1): 3, (0, 2): 3, (0, 3): 3, (0, 4): 3, (0, 5): 2, (1, 0): 3, (1, 1): 4, (1, 2): 4, (1, 3): 4, (1, 4): 4, (1, 5): 3, (2, 0): 3, (2, 1): 4, (2, 2): 4, (2, 3): 4, (2, 4): 4, (2, 5): 3, (3, 0): 3, (3, 1): 4, (3, 2): 4, (3, 3): 4, (3, 4): 4, (3, 5): 3, (4, 0): 3, (4, 1): 4, (4, 2): 4, (4, 3): 4, (4, 4): 4, (4, 5): 3, (5, 0): 2, (5, 1): 3, (5, 2): 3, (5, 3): 3, (5, 4): 3, (5, 5): 2}
网络平均度: 3.3333333333333335



由度分布图可知，二维方格图所有节点只有三种度值：2、3、4，其中顶点节点度为2，边界点（除顶点外）度为3，中间节点度为4。

Task2:

网络直径为：10

网络的平均路径长度 4.0

网络的密度为：0.09523809523809523

平均聚类系数 0.0
全局聚类系数 0
节点横截性 {(0, 0): 0.005331465919701215, (0, 1): 0.04085434173669469, (0, 2): 0.05591503267973858, (0, 3): 0.05591503267973858, (0, 4): 0.04085434173669469, (0, 5): 0.005331465919701215, (1, 0): 0.04085434173669468, (1, 1): 0.11179271708683473, (1, 2): 0.14595238095238094, (1, 3): 0.14595238095238094, (1, 4): 0.11179271708683473, (1, 5): 0.04085434173669468, (2, 0): 0.055915032679738554, (2, 1): 0.145952380952381, (2, 2): 0.1915499533146593, (2, 3): 0.19154995331465927, (2, 4): 0.14595238095238097, (2, 5): 0.05591503267973857, (3, 0): 0.05591503267973857, (3, 1): 0.14595238095238094, (3, 2): 0.1915499533146592, (3, 3): 0.1915499533146592, (3, 4): 0.14595238095238097, (3, 5): 0.05591503267973858, (4, 0): 0.04085434173669468, (4, 1): 0.11179271708683473, (4, 2): 0.145952380952381, (4, 3): 0.14595238095238094, (4, 4): 0.11179271708683473, (4, 5): 0.04085434173669468, (5, 0): 0.005331465919701215, (5, 1): 0.04085434173669469, (5, 2): 0.05591503267973857, (5, 3): 0.05591503267973857, (5, 4): 0.04085434173669467, (5, 5): 0.005331465919701215}
网络横截性 0.08823529411764709

二维方格图中，各节点聚类系数均为0，因为每个节点的邻居节点间均不存在路径。
